CSE 598 008
EECS 498 003
Lab 6

Tianyiming (Henry) Jing

October 9, 2026

Based on slides by Ivan Wei (Fall 2025).

Reminder

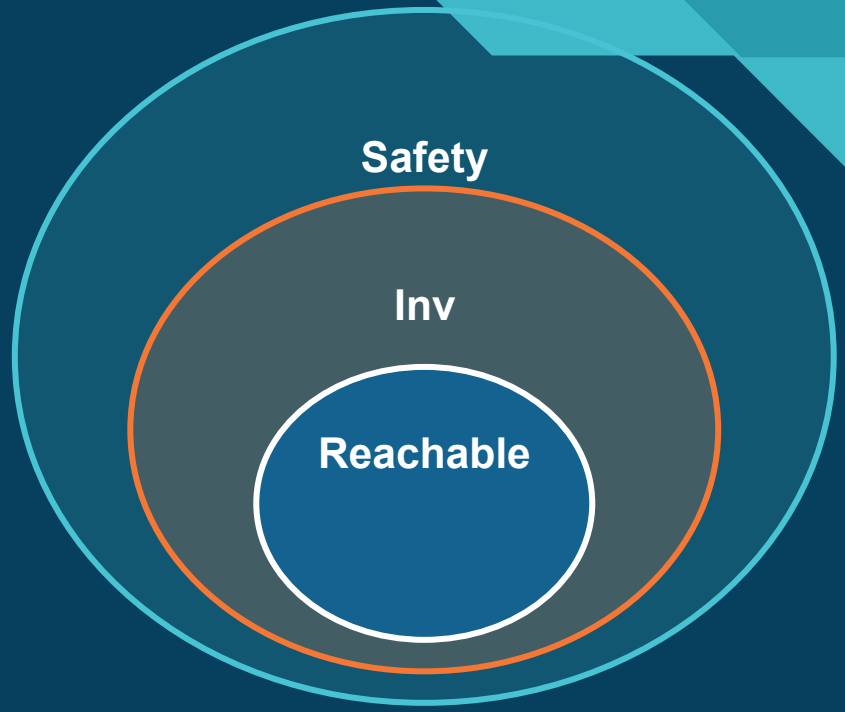
- Problem Set 3 (Chapter 5) due October 20, 11:59pm
  - It is a lot harder than PS2
- Start looking for Project 1 partners (released after PS3)
- Midterm will be on Thursday, October 22, 6pm – 8pm, FMCRB 1050
  - No lecture that day
  - Closed book; one double-sided cheat sheet allowed, 10pt minimum
  - Covers everything up to Chapter 4 (excluding distributed systems)
  - We assume knowledge of Dafny, but no “guessing”
- Midterm evaluations are out, please take some time to complete them

Your Feedback

- Thank you for your answers last week!
- Which labs do you prefer?
  - Key concepts first (Labs 4 and 5): 9
  - Exercises directly (Labs 1 to 3): 3
  - No preference: 2
- So we will keep going over key concepts first
- Today we will go over some of the questions you wrote

Why Both Safety and Inv?

- `Safety` is the property we care about
  - It is part of the spec: short and readable
  - To trust the result, a reader only needs to check `Safety`
- `Inv` is a tool for the proof
  - It can be long and technical
  - Two people may pick different `Inv` for the same `Safety`
- Proving `Inv` alone means nothing until `Inv ==> Safety`
- `Inv` is an inductive invariant:
  - Inductive: `Inv` is closed under `Next`
  - Invariant: `Inv` holds in every reachable state



Finding Inv - Backward

- Start with `Inv` = `Safety` and ask: what could go wrong?
- Find a dangerous state: in `Inv`, but one step leaves `Inv`
- Ask why the system can never reach it, and add that reason to `Inv`
- Catch: each new conjunct must also be preserved, so it brings its own new dangerous states
- Repeat until no dangerous state is left

```
// A car: accel never changes
Init:   pos == 0, speed == 0, accel == 1
Step:   pos'   == pos + speed
        speed' == speed + accel
Safety: pos >= 0
```

- `pos = 0, speed = -1` is dangerous, so add `speed >= 0`
- New dangerous state: `speed = 0, accel = -1`, so add `accel >= 0`
- `accel` never changes: no new dangerous state, done

Finding Inv - Forward

- Imagine a friend who does not believe your `Safety`. Convince them, starting from `Init`
- Find a good property that is true at `Init` and stays true after every step
- It may lean on the properties you have already shown
- Keep going until your properties together imply `Safety`
- Your argument is the proof: the properties are `Inv`, “true at `Init`” is `InitImpliesInv`, and “stays true” is `NextPreservesInv`

```
// A car: accel never changes
Init:   pos == 0, speed == 0, accel == 1
Step:   pos'   == pos + speed
        speed' == speed + accel
Safety: pos >= 0
```

- `accel == 1` at `Init`, and no step changes it: `accel >= 0`
- Since `accel >= 0`, speed never goes down: `speed >= 0`
- Since `speed >= 0`, pos never goes down: `pos >= 0`, which is `Safety`

Finding Inv - Which Way?

- Both ways end at the same `Inv`, from opposite ends
  - Backward: driven by what Dafny cannot prove, good when a lemma fails
  - Forward: driven by why the system works, good for a first guess
- Use both: guess forward, then fix backward
  - Lab 5, backward: `[-5, 0]` is safe, but one transfer gives `[-4, -1]`, so rule it out
  - Lab 5, forward: the total starts at 0 and every transfer keeps it
  - Both give `Sum(wealths) == 0`
- Too strong? Every conjunct must hold in every reachable state
  - If `InitImpliesInv` fails, a conjunct is too strong or just wrong
- Keep it easy to write: one fact per conjunct, and name long ones with a helper predicate

Loop Invariants

```
method SumUpTo(n: nat) returns (s: nat)
    ensures s == n * (n + 1) / 2
{
    s := 0;
    var i := 0;
    while i < n
        invariant i <= n
        invariant s == i * (i + 1) / 2
    {
        i := i + 1;
        s := s + i;
    }
}
```

- A loop invariant is an inductive invariant, with one loop iteration as the step
- `Init ==> Inv`: it holds before the loop
- `Inv && Next ==> Inv'`: one iteration (with the guard true) keeps it
- `Inv ==> Safety`: when the loop exits, `Inv && !guard` must give the `ensures`
- Without `i <= n`, Dafny cannot get `i == n` after the loop: same “too weak” problem as before

What Dafny Needs

- Calling a function, method, or lemma: its `requires` must hold first
- After a call, Dafny only knows its `ensures`, not how its body works
  - Function bodies are different: Dafny can unfold them one level
- Induction: call the lemma on a smaller input (that `decreases`)
- Loops: an `invariant` (previous slide) and a `decreases`
- `forall` facts: used only when Dafny sees the trigger term, so mention it in an `assert`
- `exists` goals: give a witness, e.g. `assert P(3);`
- `Next` in JNF: `var step :| NextStep(...)`, then `match step` (one case per action)
- On paper, ask the same question at each line: what does Dafny know here?

Question

- What would you like us to review before the midterm?
- What do you think others in this class would like us to review?
- Write your answers on the piece of paper

Proof Exercise

- Any questions about last week's problems? We can discuss them
- This week: The Bridges of Euleria
- Starter code: `lab06.dfy` on the course GitHub

Extra - Surprise Exam Paradox

- A teacher says: “There will be a surprise exam one day next week. You will not know which day until it happens.”
- Which day could the exam be on?

Extra - Surprise Exam Paradox

- Surprise: on the evening before the exam, you cannot be sure the exam is tomorrow, since a later day is still possible
- The students' reason:
  - It cannot be Friday: after Thursday passes, we would know
  - Then Thursday is the last possible day, so it cannot be Thursday either
  - Same for Wednesday, Tuesday, and Monday: there is no exam!
- The exam comes on Wednesday. Everyone is surprised
  - Don't worry, our midterm is no surprise: October 22
- Dafny code: `surprise_exam.dfy` in the extra folder on the course GitHub

https://en.wikipedia.org/wiki/Unexpected_hanging_paradox